

Week 3

C237:

Software Application Development

LESSON 05:

Introduction to NodeJS



Introduction to NodeJS

What is Node.JS?

- Node.js uses **JavaScript** as its main programming language
- It is a **runtime environment** that runs JavaScript **outside the browser**
- It allows developers to build **server-side applications** using JavaScript

Who uses Node.JS?

- Used by large-scale platforms for **speed, scalability, and handling many users**



- Uses Node.js for a **fast, responsive user experience**
- Handles **high traffic** with asynchronous operations ((multiple requests handled at the same time))



- Uses Node.js to **improve performance**
- Enables **faster development and efficient data processing**

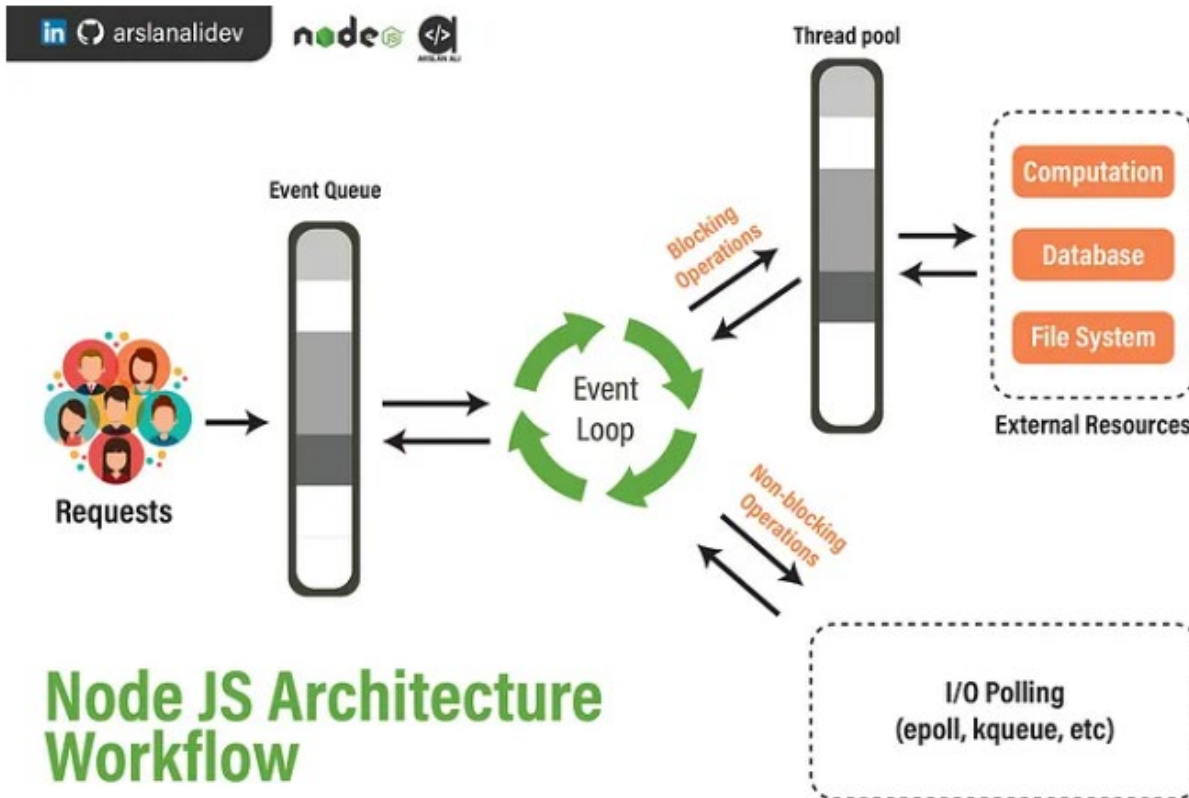
Why Node.JS?

- **JavaScript on Server**
 - Traditionally used in browsers (client-side)
 - Node.js allows JavaScript to run on the **server**.
- **Event-Driven and Asynchronous**
 - Uses event-driven architecture
 - **Non-blocking:** handles multiple requests without waiting
 - Improves **performance and scalability**.
- **Fast Execution**
 - Enables **high-speed execution**

What can Node.JS do?

- Generate **dynamic page content**
- Handle **files on the server** (read, write, delete)
- Process **form data**
- Manage database operations (Add, Update, Delete)

NodeJS Architecture



Requests

- Requests coming from client side.

Event Queue

- Stores the request and passes them one by one into the event loop

Event Loop

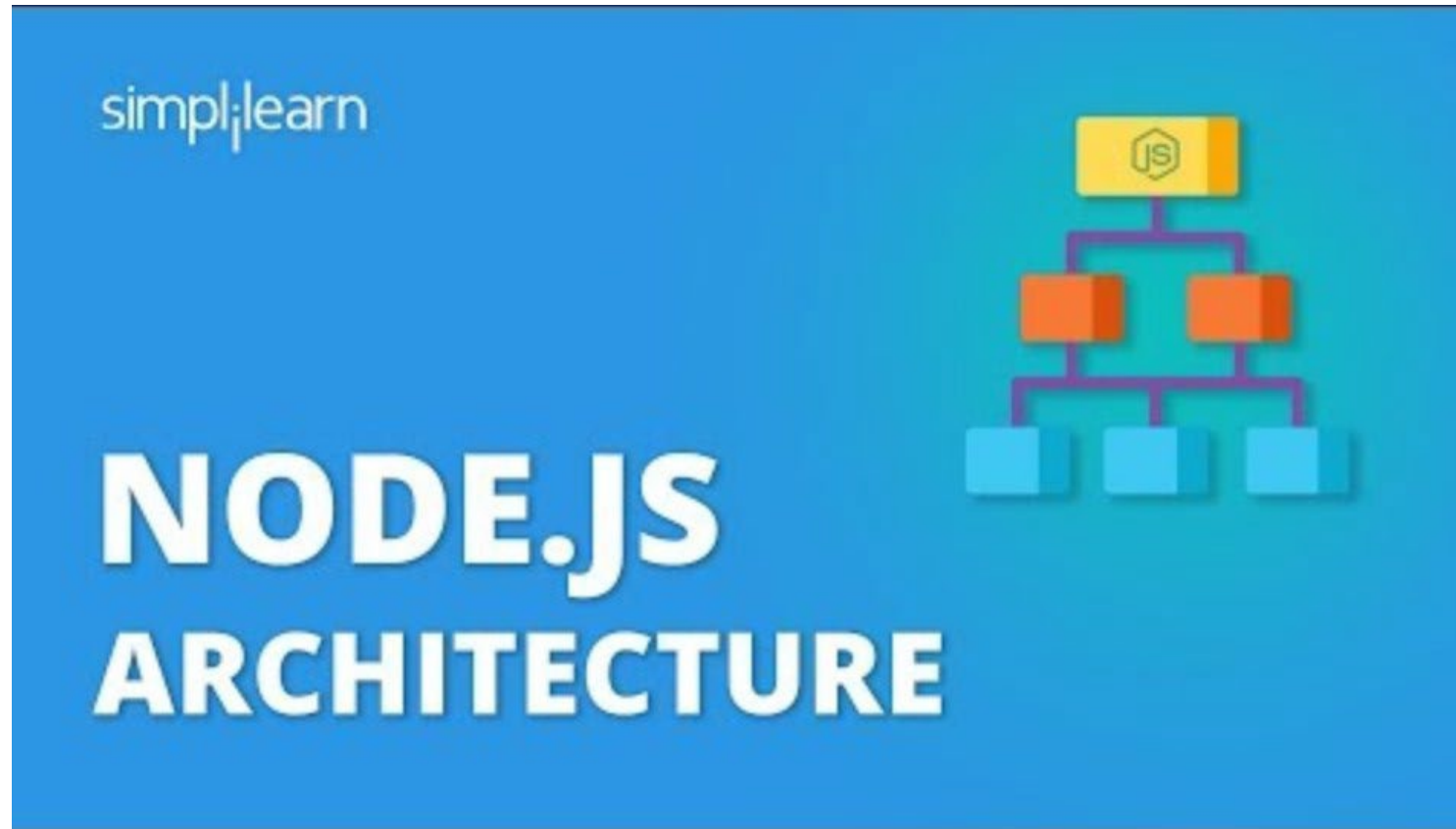
- Receives the request, process the request and returns the response to the particular client.

Thread Pool

- Consists of the Available threads in NodeJS server which will take some tasks to complete the client request.

An Overview of How the Web Works

Do watch this video when you can—it'll help you better understand the topic.



<https://youtu.be/zFRuGk52X-c?si=CrtYIfD0Xo9mXzwQ>



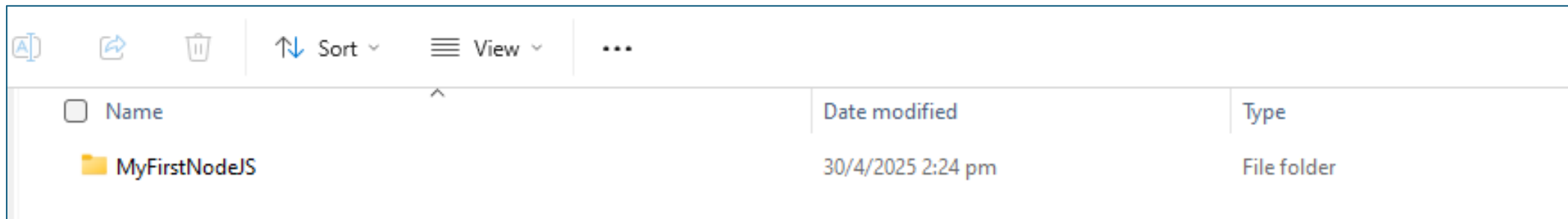
NodeJS Environment

NodeJS Packages

- NodeJS packages are **code libraries** provided by third party programmers to solve specific problems.
- **NPM - node package manager**
 - Used to **install update and remove packages** for our application

Setting up Visual Studio Code for Node.js

- **Open Visual Studio Code**
 - Launch VS Code on your laptop
- **Create a New Folder:**
 - Create a L05 folder in your C237 folder.
 - Create a new folder (e.g. **“MyFirstNodeJS”**) in L05 folder.

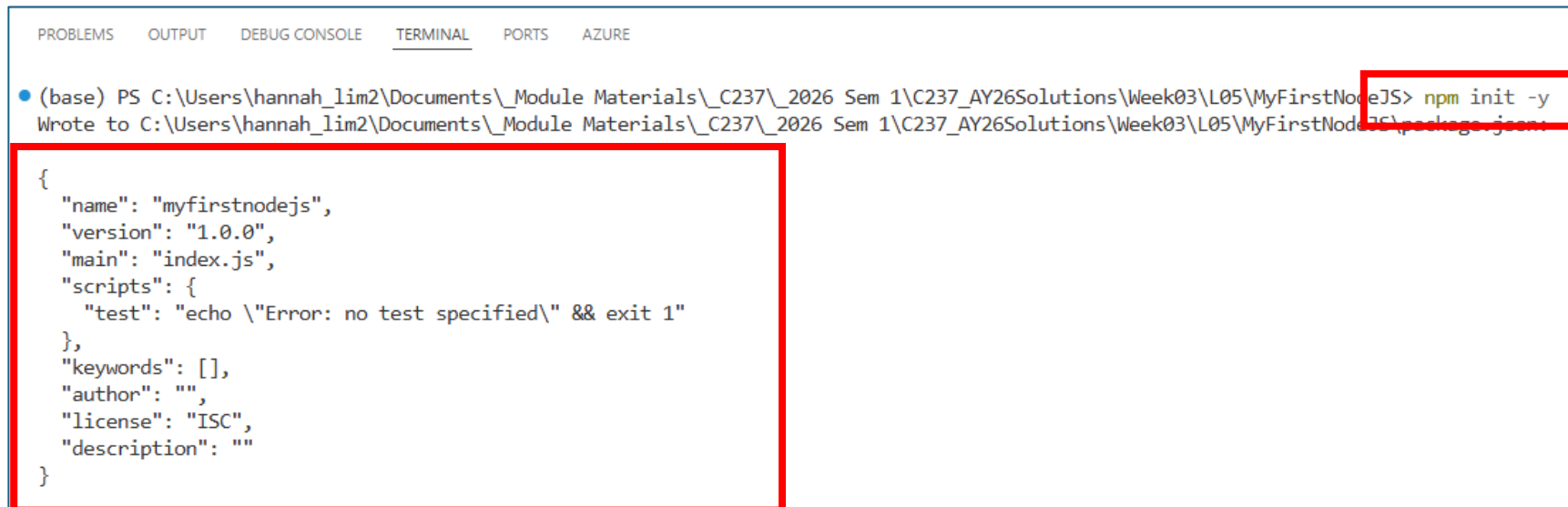


The screenshot shows a file explorer interface with a toolbar at the top containing icons for opening, sharing, deleting, sorting, and viewing. Below the toolbar is a table listing files and folders. The table has three columns: 'Name', 'Date modified', and 'Type'. A single entry is visible: a folder named 'MyFirstNodeJS' with a date modified of '30/4/2025 2:24 pm' and a type of 'File folder'.

☐ Name	Date modified	Type
 MyFirstNodeJS	30/4/2025 2:24 pm	File folder

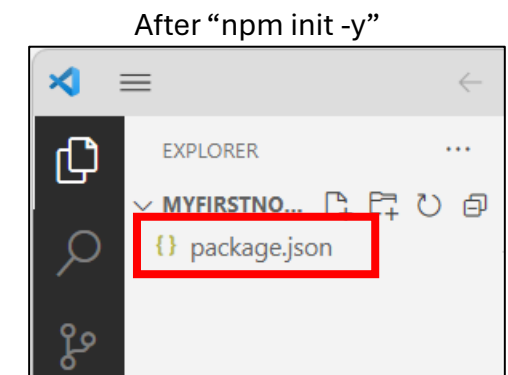
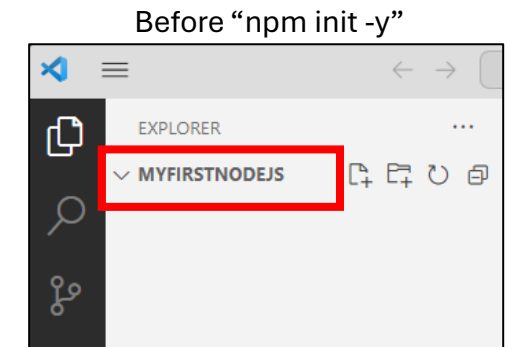
Setting up Visual Studio Code for Node.js

- **Open the folder which you just created in VS Code:**
 - Open VS Code and use the "File" menu to open your newly created folder.
 - Open a terminal within VS Code (Terminal > New Terminal).
 - Run the following command to create a package.json file for your project:
 - `npm init -y`



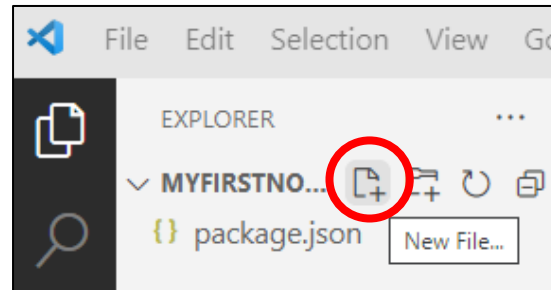
```
(base) PS C:\Users\hannah_lim2\Documents\Module Materials\C237_2026 Sem 1\C237_AY26Solutions\Week03\L05\MyFirstNodeJS> npm init -y
Wrote to C:\Users\hannah_lim2\Documents\Module Materials\C237_2026 Sem 1\C237_AY26Solutions\Week03\L05\MyFirstNodeJS\package.json:

{
  "name": "myfirstnodejs",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

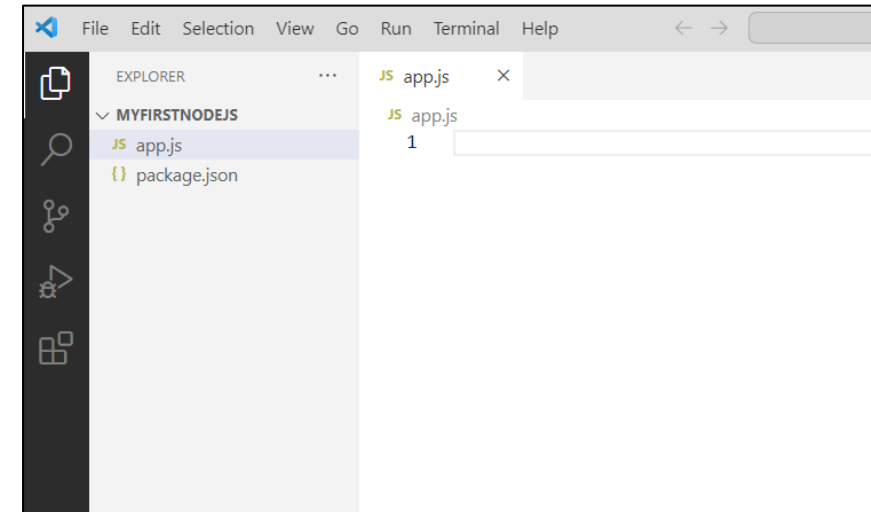


Creating a new JS file

- **Create a New JavaScript File:**
 - Inside the folder, create a new file with a '.js' extension (e.g. app.js)
 - Click on the new file icon



- Double click on the file in VS Code to open the file.



Creating a new JS file

- Write your First Node.js Code:
 - In **app.js**, type or paste the following code and save the file.

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Hello, World!\n');
});

const PORT = 3000;

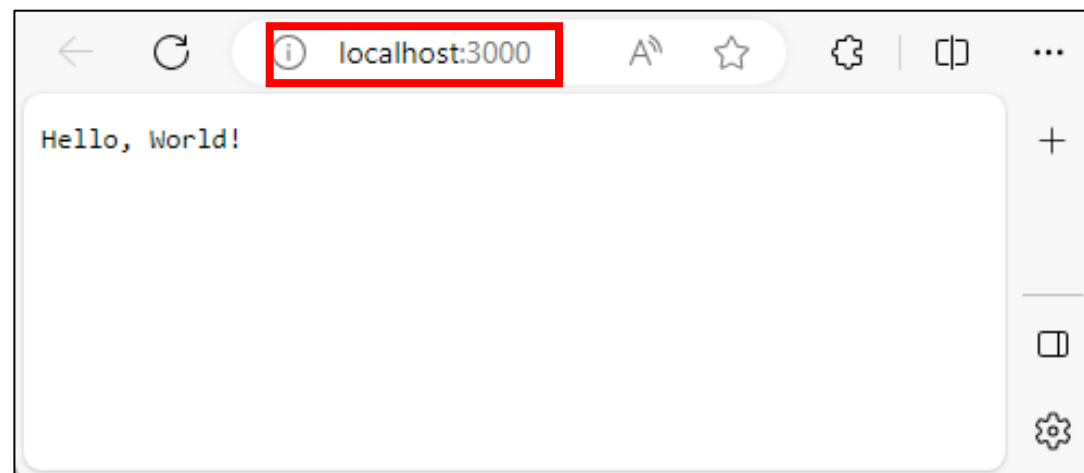
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});
```

- Save the file

Creating a new JS file

- **Write your First Node.js Code (continued):**
 - Open the terminal in VS Code
 - Run your first Node.js app by typing the following command in your terminal: **node app.js**
 - You should see the message “Server running at <http://localhost:3000/>”

```
(base) PS C:\Users\hannah_lim2\Documents\Module Materials\C237_2026 Sem 1\C237_AY26Solutions\Week03\L05\MyFirstNodeJS> node app.js  
Server running at http://localhost:3000/
```



Basic Node.JS script

- This script creates a **simple HTTP server** using the **built-in http module** and listens on **port 3000**. When you access **http://localhost:3000** in your web browser, it will respond with "Hello, World!".

```
JS app.js  X
JS app.js > ...
1  const http = require('http');
2
3
4  const server = http.createServer((req, res) => {
5    |   res.end('Hello, World!');
6  | });
7
8  const PORT = 3000;
9
10 server.listen(PORT, () => {
11   |   console.log(`Server running at http://localhost:${PORT}/`);
12 | });
```


Basic Node.JS script

```
JS app.js  X
JS app.js > ...
1  const http = require('http');
2
3
4  const server = http.createServer((req, res) => {
5    |   res.end('Hello, World!');
6    | });
7
8  const PORT = 3000;
9
10 server.listen(PORT, () => {
11   |   console.log(`Server running at http://localhost:${PORT}/`);
12   | });
```

http module

- is a built-in module in Node.js for creating servers

createServer()

- Handles incoming requests
- (req, res) -> request & response

PORT

- Defines server port (e.g. 3000)

listen()

- Starts the server
- Runs on the specified port

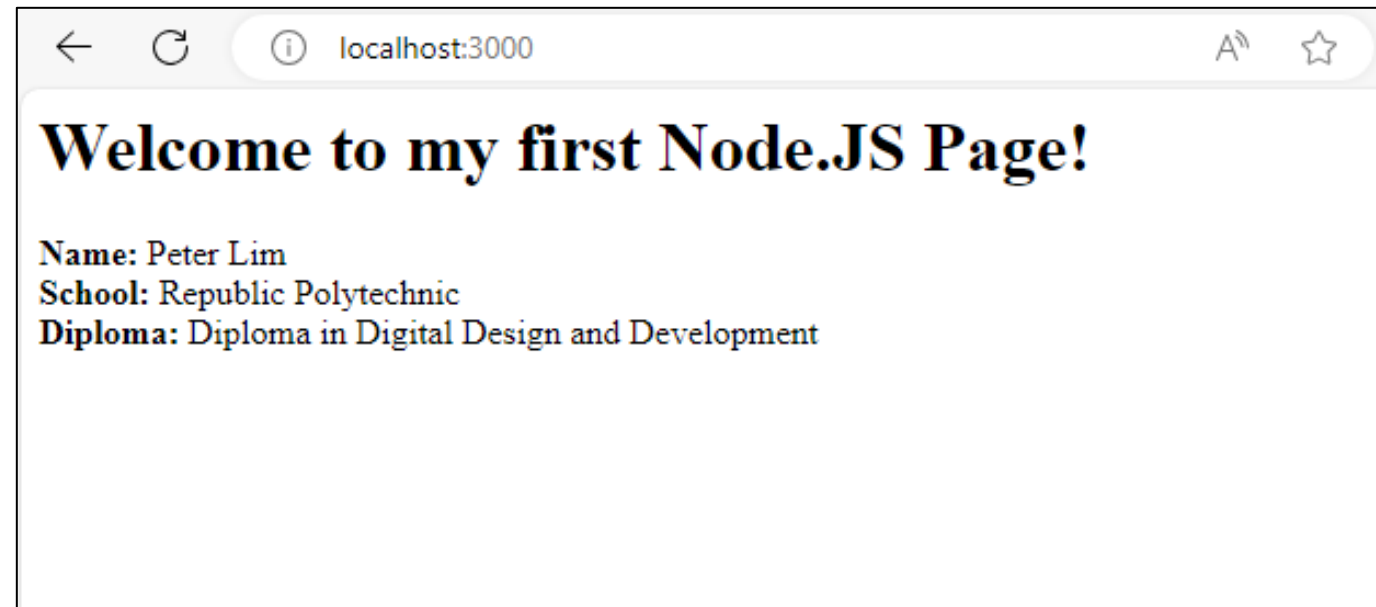
res.writeHead() sets the response status and headers.

res.write() sends parts of the response body and can be called multiple times.

res.end() ends the response and can optionally send the final chunk of data.

Let's Try!

- **Make Changes and Save:**
 - Modify the code in **app.js** to include information about yourself.
- **Run the project:**
 - To refresh your page, **stop your server (Ctrl + C)**
 - Run your first Node.js app by typing the following command in your terminal: **node app.js**
 - You should see the message “Server running at <http://localhost:3000/>”
- **Sample Screenshot:**





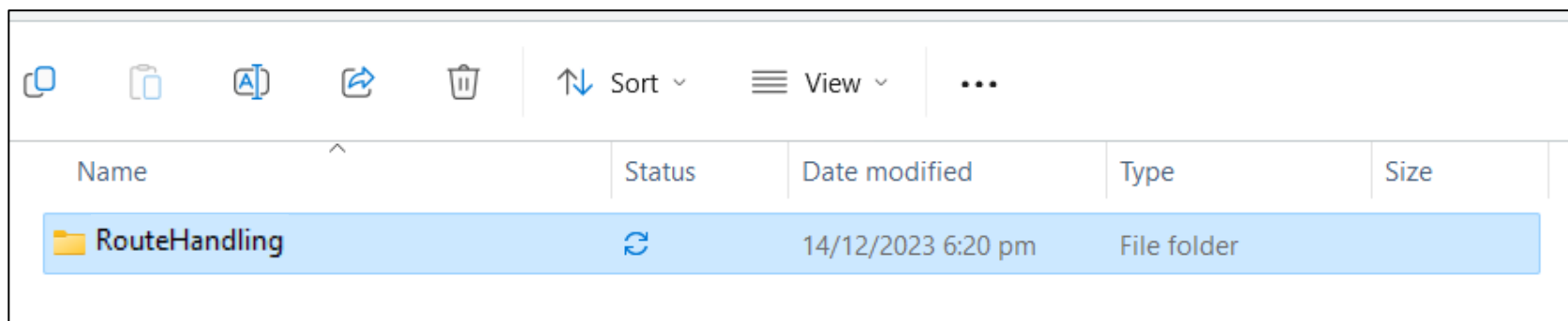
NodeJS App Routes Handling

Node.JS script – Routes

- **Routes**
 - Paths (URLs) that the server can handle
- **How Routing Works?**
 - User enters a URL
 - Server matches it to a route
 - Runs the corresponding code
 - Sends a response
- **Example**
 - / - leads to Home
 - **/about** - leads to About Page

Setting up Visual Studio Code for Node.js

- **Open Visual Studio Code**
 - Launch VS Code on your laptop
- **Create a New Folder:**
 - Create a new folder (e.g. "RouteHandling") in the L05 folder which you created earlier.



The screenshot shows a file explorer interface with a toolbar at the top containing icons for copy, paste, search, share, delete, sort, view, and a menu. Below the toolbar is a table with columns: Name, Status, Date modified, Type, and Size. A single row is highlighted in blue, representing a folder named 'RouteHandling'.

Name	Status	Date modified	Type	Size
 RouteHandling		14/12/2023 6:20 pm	File folder	

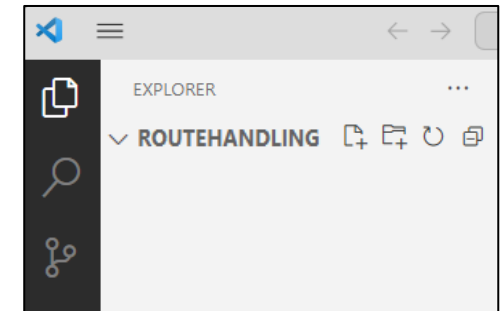
Setting up Visual Studio Code for Node.js

- **Open the folder which you just created in VS Code:**
 - Open VS Code and use the "File" menu to open your newly created folder.
 - Open a terminal within VS Code (Terminal > New Terminal).
 - Run the following command to create a package.json file for your project:
 - **npm init -y**

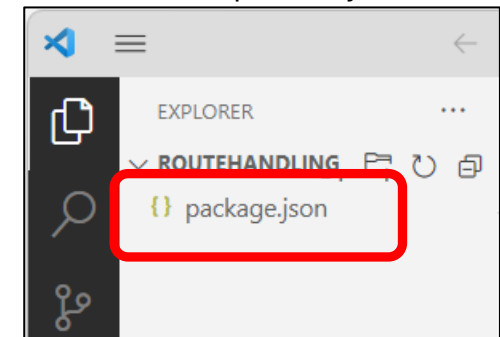
```
(base) PS C:\Users\hannah_lim2\Documents\Module Materials\C237_2026 Sem 1\C237_AY26Solutions\Week03\L05\RouteHandling> npm init -y
Wrote to C:\Users\hannah_lim2\Documents\Module Materials\C237_2026 Sem 1\C237_AY26Solutions\Week03\L05\RouteHandling\package.json:

{
  "name": "routehandling",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Before "npm init -y"

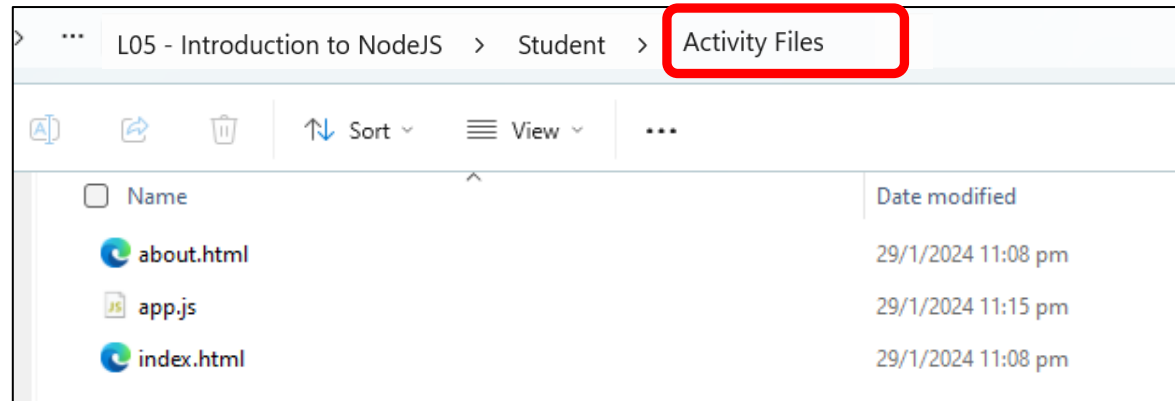


After "npm init -y"

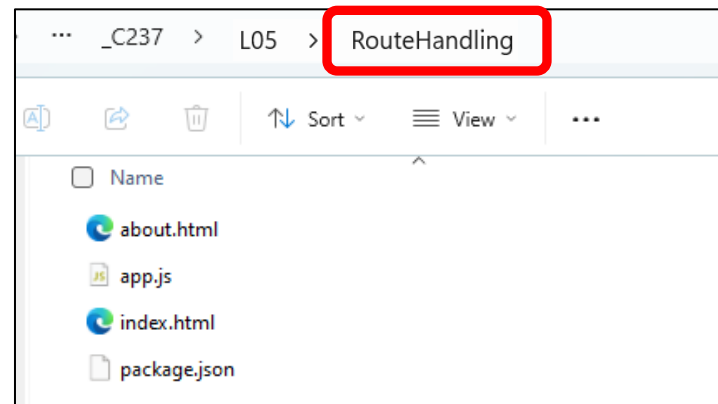


Write your Node.js Code

- **Copy your Node.js Code:**
 - In your student folder, you will see a folder named “**Activity Files**”.



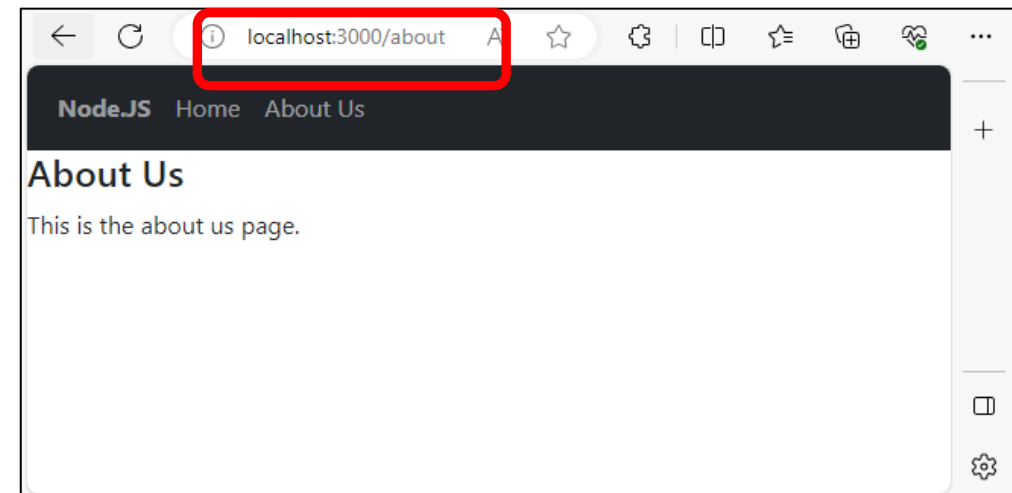
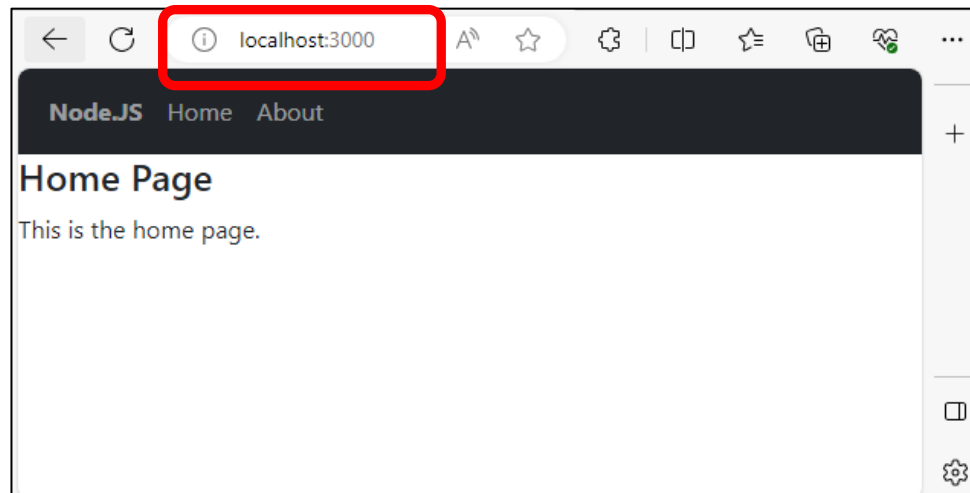
- Copy the following files and paste it into your “**RouteHandling**” folder.
 - app.js
 - about.html
 - index.html



Run your Node.js file

- **Run your Node.js Code:**
 - Open the terminal in VS Code
 - Run your Node.js app by typing the following command in your terminal: **node app.js**
 - You should see the message “Server running at <http://localhost:3000/>”

```
(base) PS C:\Users\hannah_lim2\Documents\_Module Materials\_C237\_2026 Sem 1\C237_AY26Solutions\Week03\L05\RouteHandling> node app.js  
Server running at http://localhost:3000/
```

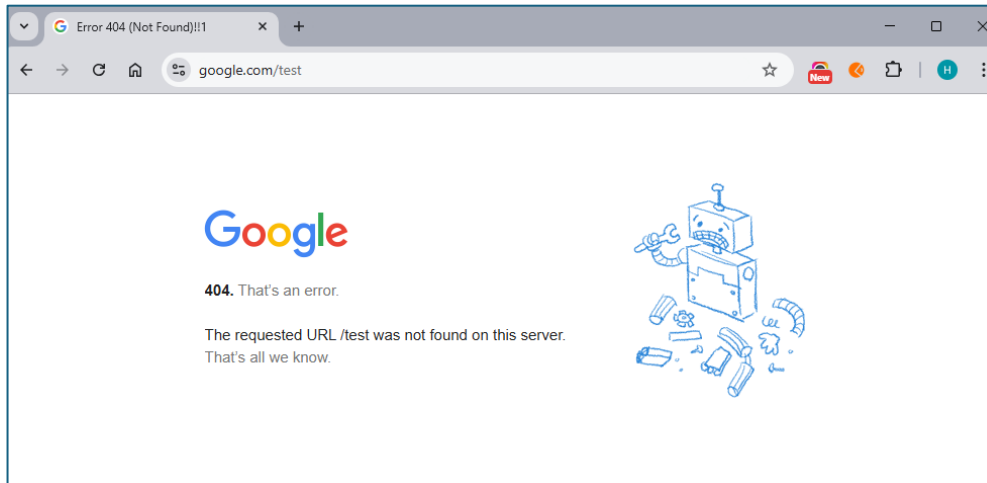


Discussion

- On localhost try changing the route
 - E.g instead of <http://localhost:3000/about> change to <http://localhost:3000/login>.
 - What happens?
- Try accessing this link: <http://google.com/test>.
 - What do you see?

Discussion

- <http://google.com/test>
 - The page shows the 404 Not Found error
 - This means the route does not exist on the server.



- What users see depends on **how routes are defined** in your code
 - If the route exists → page loads (**200 OK**)
 - If the route does not exist → error page (**404 Not Found**)

Node.JS script – Routes Handling

```
JS app.js  X  <> about.html  <> index.html

JS app.js > notFoundHandler
1  const http = require("http");
2  const url = require("url");
3  const fs = require("fs");
4
5  const routeHandlers = {
6    "/": homeHandler,
7    "/about": aboutHandler,
8  };
9
10 const server = http.createServer((req, res) => {
11   const parsedUrl = url.parse(req.url, true);
12   const path = parsedUrl.pathname;
13   const handler = routeHandlers[path] || notFoundHandler;
14   handler(req, res);
15 });
```

- This code demonstrates a basic server with **routing capabilities** in Node.js, **handling different routes** and **responding with corresponding content**.

Node.JS script – Routes Handling

```
JS app.js    X    <> about.html    <> index.html

JS app.js > ...
1  const http = require("http");
2  const url = require("url");
3  const fs = require("fs");
```

Core Modules

- **http module - server**
 - Create web servers
 - Handle requests & responses
- **url module – URL handling**
 - Work with URLs
 - Extract parts (e.g. /about)
- **fs (File System) module – file handling**
 - Read/Write files

Node.JS script – Routes Handling

```
5  const routeHandlers = {
6    "/": homeHandler,
7    "/about": aboutHandler,
8  };
9
10 const server = http.createServer((req, res) => {
11   const parsedUrl = url.parse(req.url, true);
12   const path = parsedUrl.pathname;
13   const handler = routeHandlers[path] || notFoundHandler;
14   handler(req, res);
15 });
16
17 function homeHandler(req, res) {
18   res.writeHead(200, { "Content-Type": "text/html" });
19   fs.createReadStream("index.html").pipe(res);
20 }
21
22 function aboutHandler(req, res) {
23   res.writeHead(200, { "Content-Type": "text/html" });
24   fs.createReadStream("about.html").pipe(res);
25 }
```

- **routeHandlers**
 - Maps URL - function
- **url.parse()**
 - Gets info from request URL
- **pathname:**
 - Extracts the path (e.g. /about)
- **routeHandlers[path]**
 - Finds matching handler

Node.JS script – Routes Handling

```
17 function homeHandler(req, res) {
18   res.writeHead(200, { "Content-Type": "text/html" });
19   fs.createReadStream("index.html").pipe(res);
20 }
21
22 function aboutHandler(req, res) {
23   res.writeHead(200, { "Content-Type": "text/html" });
24   fs.createReadStream("about.html").pipe(res);
25 }
26
27 function notFoundHandler(req, res) {
28   res.writeHead(404, { "Content-Type": "text/html" });
29   res.end("Not Found");
30 }
31
32 const PORT = 3000;
33 server.listen(PORT, () => {
34   console.log(`Server running at http://localhost:${PORT}/`);
35 });
```

Handler functions:

- Each route is a function
- Handles request & sends response

routeHandlers[path]:

- homeHandler -> sends index.html
- aboutHandler -> sends about.html
- notFoundHandler -> sends 404

Routing Logic

- / → **200 OK** → Home
- /about → **200 OK** → About
- Others → **404 Not Found**

Other Common Status Codes

- **201 Created** → New resource created
- **400 Bad Request** → Invalid request
- **401 Unauthorized** → Not logged in
- **403 Forbidden** → No permission
- **500 Internal Server Error** → Server error

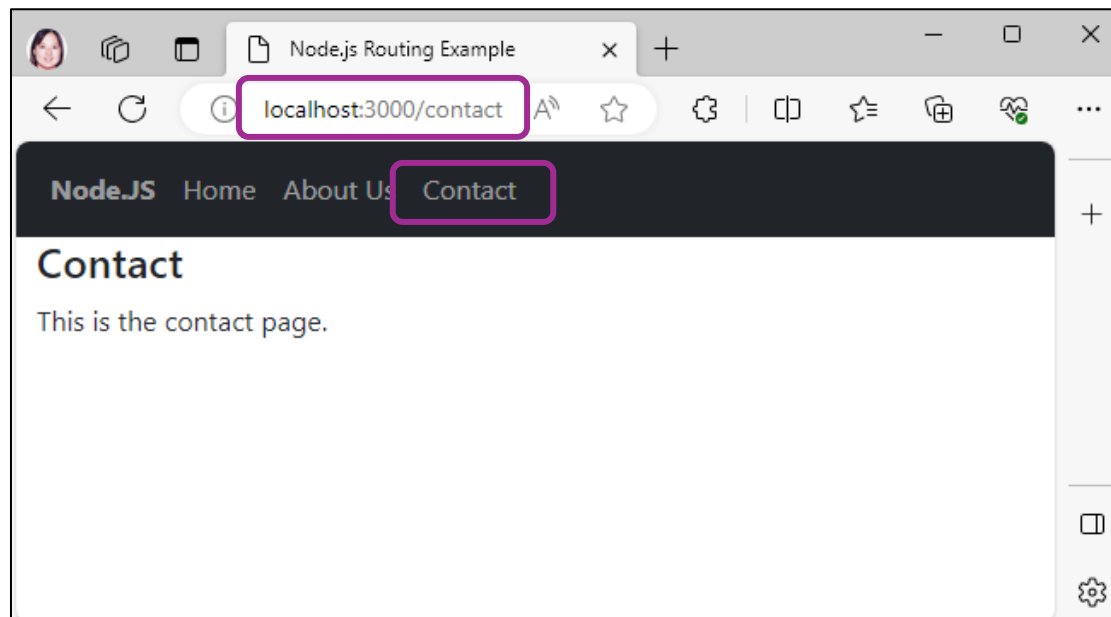
For bigger apps → use **Express.js**

Let's Try! – NodeJS Routes Handling

- **Task**
 - Update **app.js** to add a new route for **Contact**
 - Create a new **HTML file** named **contact.html**
 - Add a **hyperlink** so users can open the Contact page.
- **Use AI as support:**
 - You may ask AI for help to:
 - create the contact.html page
 - suggest code for the new route
 - suggest how to add the hyperlink/
- **Use AI responsibly**
 - Do not copy blindly
 - Read and test the code before using it
 - Make sure you understand what was added or changed

Let's Try! – NodeJS Routes Handling

- **Run your project:**
 - Run your Node.js app by typing the following command in your terminal: **node app.js**
 - You should see the message “Server running at <http://localhost:3000/>”
- **Sample Screenshot:**



Let's Try! – Did you get it right?

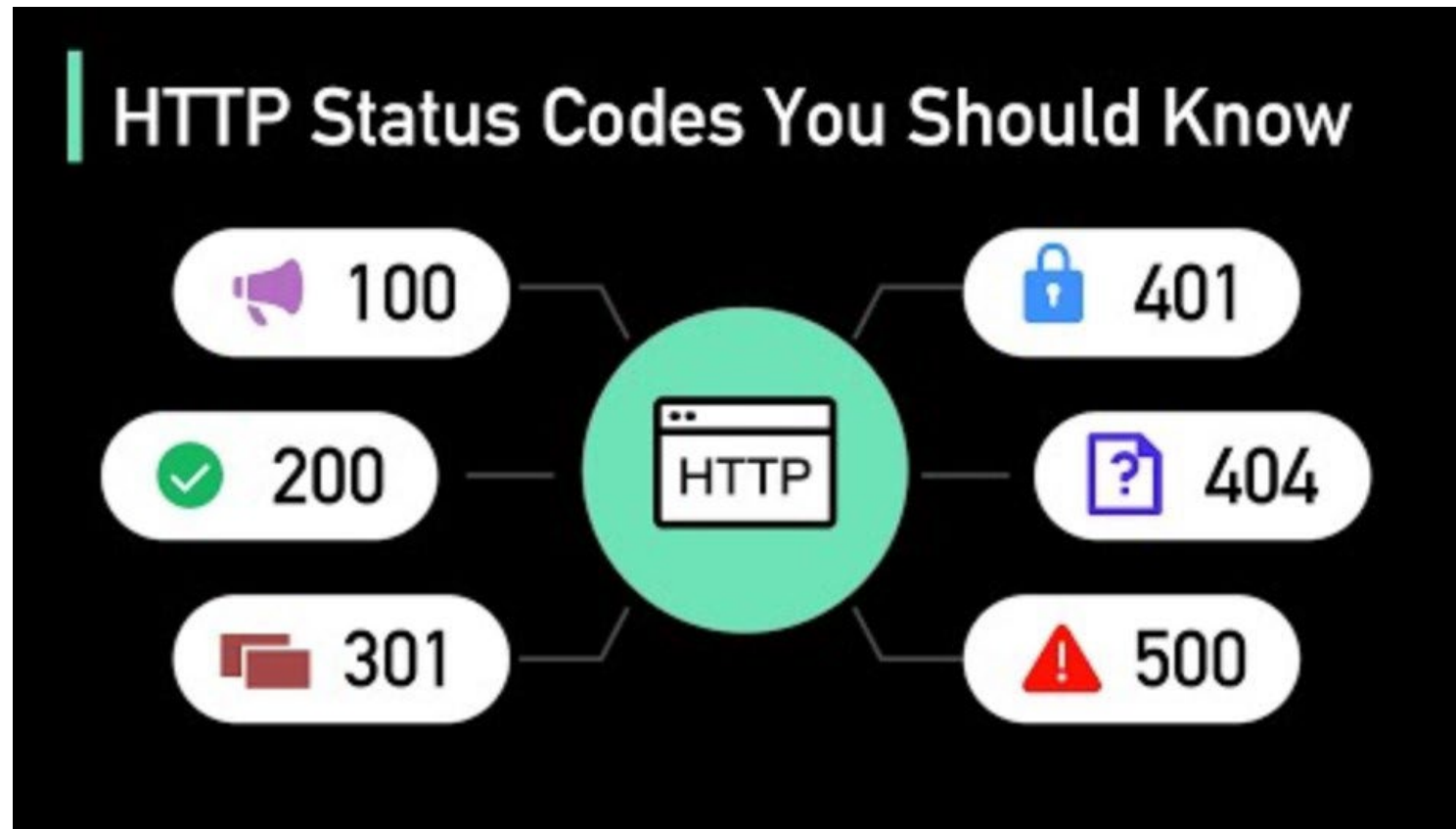
```
18 ✓ function homeHandler(req, res) {  
19   res.writeHead(200, { "Content-Type": "text/html" });  
20   fs.createReadStream("index.html").pipe(res);  
21 }  
22  
23 ✓ function aboutHandler(req, res) {  
24   res.writeHead(200, { "Content-Type": "text/html" });  
25   fs.createReadStream("about.html").pipe(res);  
26 }  
27  
28 ✓ function contactHandler(req, res) {  
29   res.writeHead(200, { "Content-Type": "text/html" });  
30   fs.createReadStream("contact.html").pipe(res);  
31 }  
32  
33 ✓ function notFoundHandler(req, res) {  
34   res.writeHead(404, { "Content-Type": "text/html" });  
35   res.end("Not Found");  
36 }
```

```
const routeHandlers = {  
  "/": homeHandler,  
  "/about": aboutHandler,  
  "/contact": contactHandler  
};
```

```
<nav class="navbar navbar-expand-sm bg-dark navbar-dark">  
  <div class="container-fluid">  
    <ul class="navbar-nav">  
      <li class="nav-item">  
        <a class="nav-link"><b>Node.JS</b></a>  
      </li>  
      <li class="nav-item">  
        <a class="nav-link" href="/">Home</a>  
      </li>  
      <li class="nav-item">  
        <a class="nav-link" href="/about">About Us</a>  
      </li>  
      <li class="nav-item">  
        <a class="nav-link" href="/contact">Contact</a>  
      </li>  
    </ul>  
  </div>  
</nav>
```

HTTP Status Code Explained

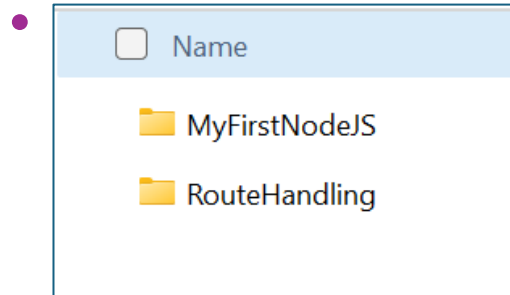
Do watch this video when you can—it'll help you better understand the topic.



<https://youtu.be/qmpUfWN7hh4?si=eeTuSsJ8iyIZ5LoS>

Deliverables

- Activity



- Upload all deliverables in a zipped folder to POLITEMall **before the next lesson.**

What have you learnt?

- Introduction to Node.JS
 - Who? What? Why? Node.JS
 - Basics of Node.JS
 - Route Handling in Node.JS
 - Routing using route handler functions
 - Setting up a NodeJS Environment

End of Lesson 05